

Docker, before the interview

The things you are most likely to be asked, in the order that is worth revising. Docker basics first, then the Spring Boot part — that is what separates you from someone who only read the docs.

SHORT ON TIME

1 · Basics

5 · Dockerfile

7 · Spring Boot

11 · Debugging

16 · Q&A

- | | | |
|---------------------|------------------|-------------------|
| 01 The basic idea | 02 How it fits | 03 Lifecycle |
| 04 Commands & setup | 05 Dockerfile | 06 Images & tags |
| 07 Spring Boot | 08 Storage | 09 Networking |
| 10 Compose | 11 Exit codes | 12 Security |
| 13 CI/CD | 14 vs Kubernetes | 15 Mistakes |
| 16 Q&A practice | 17 Debugging | 18 Practice tasks |

01

The basic idea

Why Docker exists: to stop "it works on my machine". Your JAR needs a certain JDK, some OS libraries, env vars, and a database on some port. Docker puts the app *and* everything it needs into one package that runs the same on your Mac, in CI, and in prod.

WORD	WHAT IT MEANS
Image	A read-only template: layers plus settings (env, entrypoint, ports). Like a <i>class</i> .

WORD	WHAT IT MEANS
Container	A running copy of an image = image layers + one thin writable layer + an isolated process. Like an <i>object</i> .
Dockerfile	The recipe used to build an image. Each line $\approx$ one layer.
Registry	Where images are stored — Docker Hub, ECR, GCR, Nexus, Harbor.
Volume	Storage managed by Docker that stays alive after the container is gone.
Compose	A YAML file to run many containers together (app + MySQL + Redis) with one command.

Container vs VM — asked in almost every interview

- A **VM** has **its own kernel** and a full OS. Size in GB, boots in minutes, very strong isolation.
- A **container shares the host kernel**. It is isolated using Linux **namespaces** (pid, net, mnt, uts, ipc, user), **cgroups** (CPU and memory limits), and a **union filesystem** (overlay2).
- Size in MB, starts in milliseconds, weaker isolation — all containers share one kernel.
- On Mac and Windows there *is* a small Linux VM underneath, because containers are a Linux feature.

02

How the pieces fit

```
docker CLI  --REST API--> dockerd  --> containerd  --> runc  --> your process
                        |
                        +--> registry (pull / push)
```

- **dockerd** — the daemon. Builds images and manages containers, networks and volumes. Runs as root by default, which is why *rootless mode* exists.
- **containerd** — manages the container lifecycle. Kubernetes talks to this directly.

- **runc** — creates the namespaces and cgroups and starts the process.
- **OCI** — the common standard for image format and runtime. This is why Kubernetes can run Docker-built images without Docker.
- **BuildKit** — the new build engine, now the default. Parallel stages, cache mounts, secret mounts.

03

Container lifecycle

created → running → paused → stopped (exited) → removed

- `docker run` = pull the image if it is missing, create the container (writable layer and network), then start it. Your process becomes **PID 1**.
- A container runs only as long as its **PID 1 process** runs. When the process ends, the container ends. There is no "background".
- `docker stop` sends **SIGTERM**, waits **10 seconds** (`-t`), then sends SIGKILL.
`docker kill` sends SIGKILL right away.

04

Commands, one by one

4.1 Images

COMMAND	WHAT IT DOES
<code>docker build -t myapp:1.0 .</code>	Builds an image using the Dockerfile in the current folder (<code>.</code>). <code>-t</code> gives it a name and tag.
<code>docker images</code>	Lists the images on your machine, with their size.
<code>docker pull postgres:16</code>	Downloads an image from the registry without running it.

COMMAND	WHAT IT DOES
<code>docker history myapp:1.0</code>	Shows every layer of the image and its size. Use it to find the big one.
<code>docker tag myapp:1.0 myrepo/myapp:1.0</code>	Gives the same image a second name, usually the registry name, before pushing.
<code>docker push myrepo/myapp:1.0</code>	Uploads the image to the registry.
<code>docker rmi myapp:1.0</code>	Deletes an image from your machine.

4.2 Containers

COMMAND	WHAT IT DOES
<code>docker run -d --name app -p 8080:8080 myapp:1.0</code>	Creates and starts a container. <code>-d</code> runs it in the background, <code>--name</code> gives it a name, <code>-p host:container</code> opens the port.
<code>docker run -e KEY=value ...</code>	Passes an environment variable into the container.
<code>docker ps</code>	Lists running containers.
<code>docker ps -a</code>	Lists all containers, including stopped ones and their exit code.
<code>docker logs app</code>	Prints what the app wrote to the console.
<code>docker logs -f --tail 100 app</code>	Shows the last 100 lines and keeps following new ones, like <code>tail -f</code> .
<code>docker exec -it app sh</code>	Opens a shell inside a running container – the normal way to look around. <code>-it</code> means interactive terminal.

COMMAND	WHAT IT DOES
<code>docker inspect app</code>	Prints every detail as JSON: env vars, mounts, network, exit code, and whether it was OOM-killed.
<code>docker stop app</code>	Stops it politely — SIGTERM, then waits 10 seconds.
<code>docker start app</code>	Starts a stopped container again.
<code>docker restart app</code>	Stop and start in one command.
<code>docker rm app</code>	Deletes a stopped container. <code>docker rm -f app</code> forces it while it is running.

4.3 Volumes (storage)

COMMAND	WHAT IT DOES
<code>docker volume create pgdata</code>	Creates a named volume — a folder managed by Docker.
<code>docker volume ls</code>	Lists volumes.
<code>docker volume inspect pgdata</code>	Shows where it is stored and which containers use it.
<code>docker volume rm pgdata</code>	Deletes the volume and the data inside it .

4.4 Networks

COMMAND	WHAT IT DOES
<code>docker network create app-net</code>	Creates a user-defined network. Containers on it find each other by name .
<code>docker network ls</code>	Lists networks.
<code>docker network inspect app-net</code>	Shows which containers are connected, and their IPs.
<code>docker network connect app-net app</code>	Adds an already-running container to a network.

4.5 Compose

COMMAND	WHAT IT DOES
<code>docker compose up -d</code>	Starts everything in <code>compose.yaml</code> in the background.
<code>docker compose up -d --build</code>	Same, but rebuilds the images first.
<code>docker compose ps</code>	Shows the services and their state.
<code>docker compose logs -f app</code>	Follows the logs of one service.
<code>docker compose down</code>	Stops and removes the containers and the network. Volumes stay.
<code>docker compose down -v</code>	Same, but also deletes the named volumes — your DB data is gone.

4.6 Cleanup

COMMAND	WHAT IT DOES
<code>docker image prune</code>	Removes dangling (untagged) images.
<code>docker container prune</code>	Removes all stopped containers.
<code>docker system df</code>	Shows how much disk images, containers and volumes are using.
<code>docker system prune -a</code>	Removes everything unused. Be careful with this one.

4.7 Step by step: an app, a database, a network and a volume

Step 1 — build your image.

```
cd my-spring-boot-app      # the folder with the Dockerfile
docker build -t myapp:1.0 .
docker images               # you should see myapp:1.0
```

Step 2 — create a network, so the app and the database can find each other by name.

```
docker network create app-net
```

Step 3 — create a volume, so the database data survives when the container is deleted.

```
docker volume create pgdata
```

Step 4 — start Postgres on that network, using that volume.

```
docker run -d --name postgres \  
  --network app-net \  
  -e POSTGRES_USER=appuser \  
  -e POSTGRES_PASSWORD=secret \  
  -e POSTGRES_DB=appdb \  
  -v pgdata:/var/lib/postgresql/data \  
  -p 5432:5432 \  
  postgres:16
```

- `-v pgdata:/var/lib/postgresql/data` — Postgres keeps its files in that path, so the volume is pointed there.
- `-p 5432:5432` — only needed if you also want to connect from your Mac (DBeaver, IntelliJ). Containers on `app-net` do not need it.

Step 5 — check it is ready.

```
docker ps                                # is it running?  
docker logs postgres                     # look for "database system is ready"  
docker exec -it postgres pg_isready -U appuser
```

Step 6 — start your app on the same network.

```
docker run -d --name app \  
  --network app-net \  
  -p 8080:8080 \  
  -e SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/appdb \  
  -e SPRING_DATASOURCE_USERNAME=appuser \  
  -e SPRING_DATASOURCE_PASSWORD=secret \  
  myapp:1.0
```

```
docker logs -f app
```

The URL uses `postgres` — the **container name** — and port **5432**, the port *inside* the network. Not `localhost`.

Step 7 — prove the volume works.

```
docker rm -f postgres          # delete the database container
docker run -d --name postgres --network app-net \
  -e POSTGRES_USER=appuser -e POSTGRES_PASSWORD=secret -e POSTGRES_DB=appdb \
  -v pgdata:/var/lib/postgresql/data postgres:16
```

Your tables and rows are still there, because they live in the volume, not in the container.

Step 8 — clean up when you are done.

```
docker rm -f app postgres
docker network rm app-net
docker volume rm pgdata      # this deletes the data
```

4.8 Looking inside Postgres

Open a psql session inside the container:

```
docker exec -it postgres psql -U appuser -d appdb
```

Then use these. psql commands start with a backslash; SQL ends with a semicolon.

COMMAND	WHAT IT DOES
<code>\l</code>	List all databases
<code>\c appdb</code>	Switch to the <code>appdb</code> database
<code>\dt</code>	List the tables
<code>\d users</code>	Show the columns, types and indexes of the <code>users</code> table

COMMAND	WHAT IT DOES
<code>\dn</code>	List schemas
<code>\du</code>	List users and roles
<code>SELECT * FROM users LIMIT 10;</code>	Read some rows
<code>SELECT count(*) FROM users;</code>	Count the rows
<code>\x</code>	Toggle wide output — helpful when a row has many columns
<code>\q</code>	Quit

Run one command without opening a session:

```
docker exec -it postgres psql -U appuser -d appdb -c "\dt"
docker exec -it postgres psql -U appuser -d appdb -c "SELECT * FROM users LIMIT 5;"
```

If you get "Did not find any relations", you are usually in the wrong database or the wrong schema. Check with `\c appdb` and then `\dt public.*`.

4.9 The same thing in MySQL

```
docker run -d --name mysql --network app-net \
  -e MYSQL_ROOT_PASSWORD=secret -e MYSQL_DATABASE=appdb \
  -v mysqldata:/var/lib/mysql -p 3306:3306 mysql:8

docker exec -it mysql mysql -uroot -psecret appdb
```

COMMAND	WHAT IT DOES
<code>SHOW DATABASES;</code>	List databases
<code>USE appdb;</code>	Switch database
<code>SHOW TABLES;</code>	List the tables
<code>DESCRIBE users;</code>	Show the columns of a table

COMMAND	WHAT IT DOES
SELECT * FROM users LIMIT 10;	Read some rows
exit	Quit

NOTE

Typing passwords on the command line is fine for local practice, but not for real environments — see section 12.

05

Dockerfile — the rules that matter

```
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
ARG JAR_FILE=target/*.jar          # build time only
ENV SPRING_PROFILES_ACTIVE=prod    # saved inside the image -> no secrets here!
COPY ${JAR_FILE} app.jar
RUN addgroup -S app && adduser -S app -G app
USER app                           # stop running as root
EXPOSE 8080                         # only documentation
HEALTHCHECK --interval=30s --timeout=3s --start-period=40s --retries=3 \
  CMD wget -qO- http://localhost:8080/actuator/health || exit 1
ENTRYPOINT ["java","-jar","/app/app.jar"]
CMD ["--server.port=8080"]          # default arguments, can be replaced
```

Differences you will be asked about

PAIR	ANSWER
RUN / CMD / ENTRYPOINT	RUN runs while building and makes a layer. ENTRYPOINT is the command that runs. CMD gives default arguments — anything you type after <code>docker run</code> replaces them.
exec vs shell form	Exec form <code>["java","-jar","x.jar"]</code> : java is PID 1 and gets SIGTERM, so shutdown is clean. Shell form <code>java -jar x.jar</code> runs inside <code>/bin/sh -c</code> , so <code>sh</code> is PID 1 and eats the signal — <code>docker</code>

PAIR	ANSWER
	<code>stop</code> then waits 10s and kills it. Always use <code>exec</code> form for Spring Boot.
COPY vs ADD	COPY only copies files. ADD can also download URLs and unpack tar files. Use COPY — it is simpler and predictable.
EXPOSE vs -p	EXPOSE is just information. <code>-p host:container</code> is what really opens the port. <code>-P</code> opens all EXPOSEd ports on random host ports.
ARG vs ENV	ARG works only during the build (<code>--build-arg</code>). ENV stays in the image and can be seen with <code>docker inspect</code> and image history — so keep secrets out of both.

Layer cache — the most common build question

- Every line makes a layer. Docker caches it using the line plus a hash of the files it copies.
- If one layer changes, **every layer after it is rebuilt**.
- So put the things that rarely change first: `copy pom.xml` , `run mvn dependency:go-offline` , and `copy src/` **after** that. Otherwise every code change downloads all dependencies again.
- `.dockerignore` makes the **build context** smaller — Docker sends everything in `.` to the daemon. Ignore `.git` , IDE files, `node_modules` .

TRAP

Deleting a file in a later layer does **not** make the image smaller — the file is still inside the earlier layer and can be pulled out. Install and clean up in the **same** `RUN` . Also, if your Dockerfile does `COPY target/*.jar` , don't put `target/` in `.dockerignore` .

- **Multi-stage build** — use the full JDK and Maven in stage 1, then `COPY --from=builder` only the JAR into a small JRE stage. Build tools, source code and the Maven cache never reach the final image. A Spring Boot image usually drops from about 800 MB to about 200 MB on a JRE base, and under 100 MB with alpine or jlink.
- **Base image options:** `21-jdk` (big) → `21-jre` → `21-jre-alpine` (uses musl libc, so watch out for native libraries and DNS issues) → `distroless` (no shell, smallest attack surface, hardest to debug).
- **Tags can move; digests cannot.** `latest` is only a default tag name, not "the newest". It makes builds hard to repeat and rollbacks hard. In prod use `myapp:1.4.2` or `myapp@sha256:...`.
- A **dangling image** is the old untagged layer set left behind when you rebuild the same tag (`<none>:<none>`). Clean it with `docker image prune`.

07

Spring Boot + Docker

Most of your questions will be here. Learn to explain the whole story: fat jar → layered jar → multi-stage build → buildpacks → JVM memory.

7.1 The problem with the fat JAR

A Spring Boot uber-jar is about 50 MB, and about 48 MB of it is dependencies that almost never change. `COPY app.jar` makes **one huge layer**, so even a one-line code change pushes and pulls the full 50 MB.

7.2 Layered JARs (Spring Boot 2.3+)

Spring Boot writes a `layers.idx` file that splits the JAR into 4 layers, from "rarely changes" to "changes every time":

1. `dependencies`
2. `spring-boot-loader`
3. `snapshot-dependencies`

4. `application` — your code and resources, the only one that changes in a normal deploy

So a redeploy sends **KB instead of MB**, and image pulls become very fast.

```
java -Djarmode=tools -jar app.jar list-layers      # Boot 3.3+
java -Djarmode=layertools -jar app.jar list        # Boot 2.3 - 3.2
```

7.3 The Dockerfile you should be able to write from memory

```
# ----- stage 1: build -----
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
RUN mvn -B dependency:go-offline          # cached until pom.xml changes
COPY src ./src
RUN mvn -B clean package -DskipTests

# ----- stage 2: split into layers -----
FROM eclipse-temurin:21-jre AS extractor
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
RUN java -Djarmode=tools -jar app.jar extract --layers --destination extracted
# Boot 2.3-3.2: java -Djarmode=layertools -jar app.jar extract

# ----- stage 3: run -----
FROM eclipse-temurin:21-jre
WORKDIR /app
RUN useradd -r -u 1001 spring
COPY --from=extractor /app/extracted/dependencies/ ./
COPY --from=extractor /app/extracted/spring-boot-loader/ ./
COPY --from=extractor /app/extracted/snapshot-dependencies/ ./
COPY --from=extractor /app/extracted/application/ ./
USER 1001
EXPOSE 8080
ENTRYPOINT ["java", "-XX:MaxRAMPercentage=75.0", "-jar", "application.jar"]
```

The `application.jar` in the last stage is the small launcher created by the extract step, not the uber-jar. To make the Maven step faster without adding a layer, use a BuildKit cache mount:

```
RUN --mount=type=cache,target=/root/.m2 mvn -B clean package -DskipTests
```

7.4 Buildpacks — no Dockerfile at all

```
mvn spring-boot:build-image -Dspring-boot.build-image.imageName=myorg/myapp:1.0  
./gradlew bootBuildImage
```

This uses **Cloud Native Buildpacks (Paketo)** and needs Docker running. You get a good JRE, layers, container-aware JVM memory settings and a non-root user for free. **Use it when** you don't want to maintain Dockerfiles across many services. **Use a Dockerfile when** you need extra OS packages, a company base image, or full control.

7.5 The JVM inside a container

- Since **JDK 10**, `-XX:+UseContainerSupport` is on by default, so the JVM reads the *container* limits, not the host's. On Java 8 before 8u191 the JVM saw the **host's** RAM and CPU and got killed — the classic story.
- By default max heap is only **25%** of container memory (`MaxRAMPercentage`). Set `-XX:MaxRAMPercentage=75.0` instead of a fixed `-Xmx` that ignores the limit.
- **Heap is not the same as container memory.** Metaspace, thread stacks, code cache, direct/NIO buffers and GC data sit outside the heap. Give the container about heap + 25–35%.
- `-XX:ActiveProcessorCount` matters when CPU limits are fractional — it decides GC threads, `ForkJoinPool.commonPool`, and Tomcat/Netty defaults.
- Tools: `docker stats`, `jcmd <pid> VM.native_memory`, Actuator `/actuator/metrics/jvm.memory.used`.

TRAP

Getting this wrong is the usual reason for **exit code 137**. If someone asks "why does my pod keep restarting under load?", this is the answer they want.

7.6 Config and profiles

- Spring converts env vars to properties: `SPRING_DATASOURCE_URL` → `spring.datasource.url` , `SPRING_PROFILES_ACTIVE=prod` .
- Order, highest first: command-line args → env vars / `SPRING_APPLICATION_JSON` → `application-prod.yml` → `application.yml` .
- Never put prod passwords in the image — `ENV` and `ARG` can be read with `docker history` . Pass them at runtime with `-e` , `--env-file` , Docker or Kubernetes secrets, Vault, or AWS Secrets Manager.
- The **same image** should move from dev to stage to prod. Only the env vars change.

7.7 Graceful shutdown

```
server.shutdown=graceful
spring.lifecycle.timeout-per-shutdown-phase=30s
```

And use the exec-form ENTRYPOINT so the JVM is PID 1 and really gets SIGTERM. If you start child processes, run with `--init` so signals are passed on and zombie processes are cleaned up.

7.8 Health checks

```
management.endpoint.health.probes.enabled=true    # /actuator/health/liveness & /readiness
management.endpoints.web.exposure.include=health,info,metrics,prometheus
```

TRAP

A Docker **restart policy** does not look at the **healthcheck**. An unhealthy container keeps running; only the process exiting causes a restart. Kubernetes *does* restart on a failed liveness probe, and Swarm reschedules. Also `curl` is often missing in slim images — use `wget -q0-` .

7.9 Nice things to mention

- `spring-boot-docker-compose` (Boot 3.1+) — during development the app starts your `compose.yml` itself and connects the datasource with `@ServiceConnection` .

- **Testcontainers** — start a real MySQL or Kafka in JUnit, and `@ServiceConnection` fills in the properties. Use this to answer "how do you test against a real database in CI?"

08

Data and storage

TYPE	WHAT IT IS	USE IT FOR
Named volume	A folder managed by Docker; survives <code>docker rm</code>	DB data, uploads, anything in prod
Bind mount	A specific folder on your host	Dev hot-reload, config files, certs, logs
tmpfs	Only in RAM, gone on stop	Temporary data or secrets that must not touch disk
Writable layer	The default; deleted with the container	Nothing important

```
docker run -v pgdata:/var/lib/postgresql/data postgres:16    # named volume
docker run -v "$PWD/config":/app/config:ro myapp              # bind mount, read-only
```

- **Copy-on-write:** reads come from the image layer, but the first write copies the file into the container layer. So writing a lot to the container filesystem is slow *and* the data is lost later.
- Containers should be **stateless**. Keep state in a volume or an external service. That is what makes them easy to replace and scale.
- To share data between containers, mount the same named volume — or better, talk over the network.

09

Networking

DRIVER	WHAT IT DOES
bridge (default)	Private network on one host. Containers get private IPs, and you need <code>-p</code> to reach them from the host.
user-defined bridge	Same, plus built-in DNS — containers can reach each other by name . Always prefer this.
host	No separate network. Uses the host network directly: no <code>-p</code> , fastest, no isolation. Linux only.
none	No network.
overlay	Across many hosts (Swarm).
macvlan	The container gets a real MAC and IP on your physical network.

TRAP

The **default** bridge has **no DNS**. That is the classic "my app cannot find `mysql`" bug. Compose creates a user-defined network for you, so service names work:

```
jdbc:mysql://mysql:3306/appdb .
```

- Inside the network, use the **container port** (3306), not the published host port.
- `localhost` inside a container means that container. To reach something on your Mac, use `host.docker.internal`.
- `-p 8080:8081` means hostPort:containerPort. If the host port is taken you get "port is already allocated".

10

Docker Compose

```
services:
  mysql:
    image: mysql:8
    environment:
      MYSQL_ROOT_PASSWORD: ${DB_PASSWORD}
```

```

    MYSQL_DATABASE: appdb
volumes: [ mysqldata:/var/lib/mysql ]
healthcheck:
  test: ["CMD", "mysqladmin", "ping", "-h", "localhost"]
  interval: 10s
  timeout: 5s
  retries: 10

app:
  build: .
  ports: ["8080:8080"]
  environment:
    SPRING_PROFILES_ACTIVE: docker
    SPRING_DATASOURCE_URL: jdbc:mysql://mysql:3306/appdb
    SPRING_DATASOURCE_USERNAME: root
    SPRING_DATASOURCE_PASSWORD: ${DB_PASSWORD}
  depends_on:
    mysql:
      condition: service_healthy # waits until HEALTHY, not just "started"
  restart: unless-stopped

volumes:
  mysqldata:

```

- Compose runs many containers on **one machine**. A **Dockerfile builds one image**; Compose **runs many containers** and connects them with network, volumes and env vars.
- Plain `depends_on: [mysql]` only controls **start order**. It does not wait until MySQL is ready — that is why the app crashes on startup. Fix it with `condition: service_healthy`, and also add retry in the app, because a good service should not depend on start order.
- The `version:` line at the top is no longer used in Compose v2. The command is `docker compose`; `docker-compose v1` is dead.
- Also worth knowing: `--profile debug`, `up --scale worker=3`, an `.env` file for `${VAR}`, and `docker compose config` to see the final file.

Exit codes and debugging

CODE	MEANING
0	Exited normally
1	Application error or uncaught exception
125	The Docker daemon failed — usually a wrong flag
126	Command found but not executable
127	Command not found — a typo, or no shell in a slim/distroless image
137	128+9 SIGKILL — almost always the OOM killer. Check <code>docker inspect --format '{{.State.OOMKilled}}'</code> . Can also mean a stop that timed out.
139	128+11 SIGSEGV — often a native library or the wrong CPU architecture
143	128+15 SIGTERM — a normal <code>docker stop</code>

- **Limits:** `--memory=1g --memory-swap=1g --cpus=1.5`. With no limits, one container can eat the whole host.
- **Restart policies:** `no` (default), `on-failure[:5]`, `always`, `unless-stopped`. They react to the *process exiting*, not to "unhealthy".
- **exec vs attach:** `exec` starts a new process, which is safe for debugging. `attach` connects to PID 1's input and output, where Ctrl-C can kill the container.
- **Logs:** Docker collects stdout and stderr of PID 1, so log to the console and never to a file inside the container. Logging drivers (`json-file`, `local`, `awslogs`, `fluentd`, `gelf`) send them somewhere else. Always set `max-size` and `max-file` on `json-file` or the disk fills up.

12

Security checklist

- Run as a **non-root** user — use `USER` or `--user 1001`. The default is UID 0, which is root.

- Use a small, pinned base image. Rebuild often for security patches. Scan with `docker scout cves`, Trivy or Snyk.
- **No secrets in the image** — not in `ENV`, `ARG`, or a copied `application-prod.yml`. They stay in the layers and in `docker history`. Pass them at runtime, or use a BuildKit `--mount=type=secret`.
- Use `--read-only` for the root filesystem plus a tmpfs for `/tmp`;
`--cap-drop ALL --cap-add NET_BIND_SERVICE`; `--security-opt no-new-privileges`.
- Never mount `/var/run/docker.sock` into a container you don't trust — that gives root on the host.
- Use rootless Docker on shared machines. Sign your images and keep internal ones in a private registry.

13

CI/CD and registry

- Tag images with the **git SHA** — `myapp:1.4.2`, `myapp:sha-a1b2c3d`. Then you know exactly what is running and can roll back. Use `latest` only as a shortcut.
- Pipeline: build → unit tests → build image → scan → push to registry → deploy by digest.
- Speed up CI with `--cache-from` or the BuildKit registry cache, and cache `.m2` with a cache mount.

COMMON ON MAC

An M-series Mac builds `linux/arm64`, but most servers are `linux/amd64`, so you get `exec format error`. Fix it with
`docker buildx build --platform linux/amd64,linux/arm64 --push`.

14

Docker vs Kubernetes

Docker builds and runs containers on **one machine**. Kubernetes runs them across **many machines**: scheduling, restarting failed pods, scaling, service discovery, rolling deploys, config and secrets. Compose is for development and one host; Kubernetes is for production at scale. Kubernetes removed the Docker runtime shim (dockershim) but still runs Docker-built **OCI** images through containerd. Docker Swarm is Docker's own simpler orchestrator, mostly old now.

15

Mistakes people make in interviews

- 01 Saying "containers are small VMs" — they are isolated processes sharing the host kernel.
 - 02 Thinking `EXPOSE` opens a port. It doesn't.
 - 03 Thinking deleting a file in a later layer makes the image smaller.
 - 04 Thinking `latest` means newest. It is just a tag.
 - 05 Thinking `depends_on` waits until the DB is ready.
 - 06 Thinking restart policies react to healthchecks.
 - 07 Using shell-form ENTRYPOINT, which breaks SIGTERM and graceful shutdown.
 - 08 Mixing up heap size and container memory — that is exit 137.
 - 09 Forgetting that the default bridge network has no DNS by name.
 - 10 Running `docker rm` with no volume and losing the database.
 - 11 Putting secrets in `ENV`, where anyone who pulls the image can read them.
 - 12 Building on Apple Silicon and deploying to amd64 without buildx.
-

16

Quick questions and answers

Try to answer out loud before you open each one.

[SHOW ALL](#)[HIDE ALL](#)

Basics

– What is Docker and why use it?

It packages the app with everything it needs into one image, so it behaves the same everywhere, starts fast, and uses fewer resources than VMs.

– Image vs container?

A read-only template versus a running copy of it with a writable layer. Class versus object.

– Container vs VM?

Shared kernel with namespaces and cgroups versus its own kernel. MB and milliseconds versus GB and minutes. Weaker versus stronger isolation.

– What happens on `docker run` ?

Pull the image if it is missing, create the container with a writable layer and network, then start the entrypoint as PID 1.

– Describe Docker's architecture.

CLI → dockerd over a REST API → containerd → runc, plus a registry for images.

Images and Dockerfile

– What is a layer?

A read-only set of file changes made by one Dockerfile line. Layers are cached and shared between images.

– How do you make builds faster?

Put rarely-changing lines first, copy `pom.xml` and download dependencies before `src`, use `.dockerignore`, BuildKit cache mounts and multi-stage builds.

– How do you make the image smaller?

Multi-stage build, a JRE or distroless base instead of a JDK, install and clean up in one RUN, and a `.dockerignore`. Find the big layer with `docker history`.

– CMD vs ENTRYPOINT?

ENTRYPOINT is the command, CMD is the default arguments. Args after `docker run` replace CMD; `--entrypoint` replaces ENTRYPOINT.

– What is a dangling image?

Old untagged `<none>` layers left after rebuilding a tag. Remove them with `docker image prune`.

Storage and networking

– Volume vs bind mount?

A folder managed by Docker (portable, easy to back up) versus a specific host path. Volumes for DB data, bind mounts for dev and config.

– How do two containers talk to each other?

On the same user-defined network, using the service or container name (Docker's built-in DNS) and the container port.

– Bridge vs host vs overlay?

Private network on one host / share the host's network / span many hosts.

Compose

– Compose vs Dockerfile?

A Dockerfile builds one image; Compose runs and connects many containers.

– The app starts before the DB is ready – how do you fix it?

Add a `healthcheck` on the DB and `depends_on: condition: service_healthy`, and add retry in the app (HikariCP `initializationFailTimeout`, Flyway retries).

Spring Boot

– How do you dockerize a Spring Boot app?

Multi-stage build: Maven builds the JAR, `jarmode=tools extract --layers` splits it, copy the 4 layers into a small JRE image, run as non-root, use `exec-form ENTRYPOINT`, and pass config through env vars. Or use `mvn spring-boot:build-image` and skip the Dockerfile.

– Why layered jars?

Dependencies rarely change and your code changes often, so only the small `application` layer is rebuilt, pushed and pulled.

– How do you set JVM memory in a container?

`UseContainerSupport` (on by default since JDK 10) reads the container limits. Set `MaxRAMPercentage=75` and leave room for metaspace, threads and direct buffers, or you get exit 137.

– How do you handle profiles and config?

`SPRING_PROFILES_ACTIVE` and env vars: one image, different config per environment, and secrets passed at runtime.

- **How do you get a clean shutdown?**

`server.shutdown=graceful` plus exec-form ENTRYPOINT, so the JVM is PID 1 and gets SIGTERM within the 10 second window.

- **How do you expose health?**

Actuator liveness and readiness endpoints, a Docker `HEALTHCHECK`, and in Kubernetes the liveness, readiness and startup probes.

- **How do you test against a real database?**

Testcontainers with `@ServiceConnection`, and `spring-boot-docker-compose` for local development.

Security

- **How do you secure a container?**

Non-root user, small pinned base image, regular scanning, no secrets in the image, read-only filesystem, dropped capabilities, never mount the Docker socket, and a rootless daemon.

- **How do you handle secrets?**

Only at runtime: env from a secret manager, Docker or Kubernetes secrets, Vault, or BuildKit secret mounts. Never inside a layer.

Debugging playbook

Scenario questions check whether you have a method. Say the steps out loud.

"The container exits immediately."

`docker ps -a` for the exit code → `docker logs` → `docker inspect`. Usual reasons: the app failed at startup (wrong datasource URL, missing env var), CMD was not a foreground process, a wrong path or entrypoint (127), or out of memory (137). Reproduce it with `docker run -it --entrypoint sh image` and run the command by hand.

"It works locally but not in Docker."

Usually config, not code: `localhost` instead of the service name, a missing env var or profile, a port not published, a file path or permission the non-root user can't write, DNS on the default bridge, timezone or locale, or the wrong CPU architecture.

"It gets killed under load (137)."

`docker inspect --format '{{.State.OOMKilled}}'` — if true, raise `--memory` or lower `MaxRAMPercentage`. Look for a leak with `-XX:+HeapDumpOnOutOfMemoryError -XX:HeapDumpPath=/dump` writing to a mounted volume, plus Actuator metrics.

"`docker stop` always takes 10 seconds."

PID 1 is not getting or handling SIGTERM. Use exec form, add `--init`, and turn on graceful shutdown.

"My code change didn't appear after rebuild."

Either the layer cache is stale or you are running the old tag. Try

`docker build --no-cache`, check the image ID, use `docker compose up --build`, and make sure `.dockerignore` is not excluding your source.

"The image is 1.2 GB."

`docker history` to find the big layer, then multi-stage build + JRE base + cleanup in the same RUN.

"The app can't reach MySQL."

Same network? (`docker network inspect`) Using the service name and the *container* port? Was the DB ready? Are the env vars correct? Then `docker exec -it app sh` and `nc -zv mysql 3306` .

18

Practice tasks

Do these once and you will have real examples instead of definitions.

1. Dockerize one Spring Boot service two ways — your own multi-stage Dockerfile and `spring-boot:build-image` . Compare image size and rebuild time after a one-line change.
2. Run `docker history` on your image, find the biggest layer, and remove it.
3. Compose: app + MySQL + healthcheck with `depends_on` . Stop the DB container and see what the app does. Add a named volume and check the data survives `docker compose down` and disappears with `down -v` .
4. Run with `--memory=512m` and default JVM settings, put load on it, get exit 137, then fix it with `MaxRAMPercentage` .
5. Break things on purpose: use shell-form ENTRYPOINT and time `docker stop` ; use the default bridge and watch DNS fail.
6. Try `docker buildx build --platform linux/amd64,linux/arm64` .

Sources

- Docker docs — [Get started](#), [What is a container](#), [Dockerfile best practices](#)
- Spring Boot reference — [Container images / Dockerfiles](#)
- Docker blog — [9 tips for containerizing Spring Boot](#)
- Interview question sets — [KodeKloud](#), [Dataquest](#), [DataCamp](#), [Devinterview-io](#)

